

# ByteBlower study notes: the tests and what they check

## The idea in one minute

ByteBlower sends traffic it fully controls into a device or network from one side and measures what comes out the other, so any difference is caused by the thing in between. That thing is the device under test: a modem, home gateway, Wi-Fi access point, switch or a whole access network.

Every test answers one of five questions:

| Question                                          | Measured as                                                                           |
|---------------------------------------------------|---------------------------------------------------------------------------------------|
| How much can it carry?                            | Throughput: the highest rate with acceptable loss                                     |
| Does it drop traffic?                             | Frame loss: sent minus received                                                       |
| How long do packets take, and how steady is that? | Latency, jitter and their distribution                                                |
| Does it deliver packets in order?                 | Out-of-sequence count                                                                 |
| How would a real application feel?                | TCP goodput and round-trip time, voice quality score, video buffering, gaming latency |

The traffic runs between **ports**, which are virtual hosts on the ByteBlower server, or **Endpoints**, which are an app on a real phone or laptop. A typical setup puts one port on the network side and one port or Endpoint on the customer side, then sends downstream, upstream or both.

## Two kinds of test traffic

Every test is built from one of two traffic types, and which one you pick decides what the result means.

|                   | Frame Blasting (stateless)                                 | TCP flow (stateful)                                                 |
|-------------------|------------------------------------------------------------|---------------------------------------------------------------------|
| What is sent      | UDP frames you define, at a fixed rate you choose          | A real TCP session carrying HTTP between a client and a server port |
| Who sets the rate | You. The sender never slows down                           | TCP. It backs off on loss and fills the available bandwidth         |
| What it gives     | Exact loss, latency, jitter and reordering at a known load | Goodput, throughput, round-trip time, retransmissions               |
| Repeatable?       | Yes, the same every run                                    | Roughly; results vary run to run like a real network                |
| Use it to         | Benchmark: find limits, compare devices, verify a spec     | Estimate user experience: what speed does a download get?           |
| Pass or fail      | Yes, against loss and latency thresholds                   | No. Measured and reported only                                      |

Two consequences worth remembering:

- Frame blasting can overload a device on purpose. TCP cannot, because it adapts.
- Frame size matters for frame blasting. Small frames stress packet processing; large frames stress the link. This is why throughput tests repeat for several sizes.

A TCP flow's direction comes from the HTTP method. With GET the client downloads; with PUT the client uploads. Behind NAT the inside device must start the connection, so upload tests from a device behind NAT use PUT.

## The measurements

Each measurement is taken by a specific mechanism, and knowing the mechanism tells you when to trust it.

| Measurement          | What it checks                                         | How it is taken                                                                                                 | Default pass rule in the Test Framework     |
|----------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| Frame loss           | Does the device drop traffic at this load?             | The receiver counts frames matching a filter unique to the flow; loss is sent minus received                    | Loss at most 1%                             |
| Throughput           | The highest rate the device forwards                   | Repeated frame-blasting trials at different rates until loss appears                                            | Depends on the test                         |
| Latency              | One-way delay through the device                       | The sender writes a timestamp into the last 8 bytes of each frame; the receiver subtracts it from its own clock | Latency at the 99.9th percentile under 5 ms |
| Jitter               | How much the delay varies                              | Derived from the same timestamps                                                                                | Reported, not judged                        |
| Latency distribution | The tail: how bad are the worst packets?               | A histogram of per-packet latency, drawn as CDF and CCDF up to P99.99                                           | Same percentile rule                        |
| Out-of-sequence      | Does the device reorder packets?                       | A running frame number in the payload                                                                           | Reported, not judged                        |
| TCP goodput          | Rate the application actually receives                 | Bytes delivered over the transfer time, measured at the receiving side                                          | None                                        |
| TCP throughput       | Rate on the wire including headers and retransmissions | TCP stack counters                                                                                              | None                                        |

| Measurement             | What it checks                                  | How it is taken                                                    | Default pass rule in the Test Framework                      |
|-------------------------|-------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------|
| Round-trip time         | Delay as TCP sees it                            | Time for a segment to be acknowledged; minimum, average, maximum   | None                                                         |
| Retransmissions, window | Is loss or a small window holding TCP back?     | TCP stack counters over time                                       | None                                                         |
| Voice quality           | Would a call sound good?                        | A MOS score computed from loss and delay on a simulated G.711 call | MOS at least 4                                               |
| Video buffer            | Would a stream start quickly and keep playing?  | A simulated player buffer filled by segment downloads              | Initial wait limit of 5 s; the exact rule is not spelled out |
| Congestion marks        | Does the network signal congestion the L4S way? | Count of CE-marked packets on a TCP Prague flow                    | None                                                         |
| Wi-Fi link              | Signal and association during the test          | RSSI, SSID, BSSID, channel and Tx rate read on the Endpoint        | Reported, not judged                                         |

Reading the two that people misread most:

- **Goodput versus throughput.** They should be nearly equal. A large gap means many retransmissions, so the path is losing packets.
- **Latency percentiles.** An average hides the tail. Where a flow's line crosses P99 on the CCDF graph, 99% of packets were faster than that value. Low-latency work is judged on the tail.

## The tests

The docs describe about a dozen ready-made tests, from a two-cable sanity check to full certification procedures.

| Test                          | What it does                                                                        | What it checks                                    | Pass means                                        |
|-------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| Quick self-test               | One UDP flow between two server ports cabled back to back                           | That the ByteBlower itself works                  | 100% of frames received                           |
| Loss at a fixed load          | One frame-blasting flow at a rate you set                                           | Whether the device drops traffic at that load     | Loss under your threshold, default 1%             |
| Latency under load            | The same flow with timestamps on                                                    | Delay, jitter and the latency tail at that load   | Loss under threshold and P99.9 latency under 5 ms |
| Throughput sweep (GUI wizard) | Many flows of the same frame at stepped rates in one run                            | Where loss starts as load rises                   | You read the limit off the report                 |
| RFC 2544 throughput           | An automatic search for the highest lossless rate, per frame size                   | Forwarding capacity across small to large frames  | Every size reaches its expected rate              |
| TCP speed test                | An HTTP download or upload for a set time or size                                   | Achievable speed and stability for real transfers | Not judged                                        |
| Voice call                    | Simulated G.711 RTP stream                                                          | Call quality                                      | MOS at least 4                                    |
| Gaming                        | Small UDP packets of varying size at a steady rate, as in traditional online gaming | Latency of interactive traffic                    | Latency under 5 ms                                |
| Video streaming               | Segment downloads filling a player buffer                                           | Start-up delay and stalls                         | Playback starts within the wait limit             |
| Video conference              | Combined video, voice and screen-share flows                                        | Each stream's loss, latency or MOS                | Each stream meets its own rule                    |

| Test                     | What it does                                                                     | What it checks                                                                 | Pass means                                    |
|--------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------|
| L4S                      | TCP Prague flows, or UDP with L4S markings                                       | Whether low-latency traffic stays fast while classic traffic congests the link | Not judged for TCP; thresholds for UDP        |
| TR-398 Airtime Fairness  | TCP then UDP to pairs of Wi-Fi stations                                          | Whether an access point shares airtime fairly                                  | Each station keeps over 45% of its solo speed |
| DOCSIS 4.0 ATP LL-04.40  | Classic flows with and without low-latency traffic through a cable modem         | Queue management (PIE AQM) behaviour                                           | Loss and latency per interval meet the ATP    |
| Home gateway LAN and WAN | Throughput and latency per frame size, LAN to LAN and WAN to LAN, each direction | Routing and NAT performance of a home gateway                                  | You compare against your target               |
| Wi-Fi roaming            | Low-rate UDP to Endpoints while logging signal and access point                  | When a device switches access point and what it costs                          | You read it off the graphs                    |

The voice, gaming, video, conference and L4S flows belong to the Low Latency test case, which runs any mix of them at once. That mix is the point: you see what one kind of traffic does to another.

Beyond tests, a port can act as a real host for single protocols: get an address by DHCP, join multicast groups with IGMP or MLD, open a PPPoE session, answer or send ping. The docs describe these as building blocks in the API, with no ready-made test or pass rule.

## The standard procedures, step by step

Four tests follow a published procedure; this is what each one actually does.

## RFC 2544 throughput

It finds, for each frame size, the highest rate the device forwards without exceeding a tolerated loss ([docs](#)).

1. Start at an initial bitrate and run one trial, 60 seconds in the example.
2. If loss is above the tolerance, halve the rate until a trial passes. If not, raise it until a trial fails.
3. Narrow in: after a pass, go up by half the gap between the last two rates; after a fail, go down by half.
4. Stop when the step is smaller than the wanted accuracy and the last trial passed, or at the iteration limit.
5. Repeat for each frame size. The defaults are 60, 124, 252, 508, 1020, 1276 and 1514 bytes, which are RFC 2544's 64 to 1518 without the 4-byte checksum.

The result per size is the last rate that passed. The test fails if any size ends below its `expected_bitrate`, or if an error stops the run. Read the result as a curve: a device that is fine at 1514 bytes but weak at 60 is limited by packets per second, not by bits per second.

## TR-398 Airtime Fairness

It checks that a Wi-Fi access point does not let a slow or distant station starve a good one ([docs](#)).

1. Three stations run the Endpoint app. Station 1 is always in the best position; the partner varies.
2. Three setups: station 2 close, station 2 at medium distance, and station 3 (legacy mode) close.
3. In each setup, run a TCP download to each station alone. That is its maximum.
4. Then run two UDP downloads at once for 120 seconds: station 1 at 75% of its maximum, the partner at 50%. Together they overload the access point.
5. Repeat for four access point modes: 802.11n at 2.4 GHz, 802.11ac at 5 GHz, 802.11ax at 2.4 and 5 GHz. You move stations and reconfigure by hand between runs.

It passes when each station still receives more than 45% of its own TCP maximum, and the two together exceed the total TR-398 expects for that mode.

## DOCSIS 4.0 ATP LL-04.40, part 1

It checks how a cable modem's queue management (PIE AQM) reacts to overload, alone and next to low-latency traffic ([docs](#)).

1. Procedure 1: classic flow 1 for 20 seconds, then classic flow 2 for 20 seconds. Both exceed the modem's configured rate, so loss is guaranteed.
2. Procedure 2: the same, with a low-latency flow running for the full 40 seconds.
3. Loss and latency are computed per 200 ms interval and compared with the ATP's requirements.

The operator prepares the CMTS and modem before each procedure. The docs do not publish the numeric thresholds.

## Low latency and L4S

It answers a comparative question: when the link congests, which traffic keeps its latency? You run background load and delay-sensitive flows together and compare ([docs](#)).

Two published example runs show what the output teaches:

- Basic: a UDP flow starting at 10 seconds cut a running HTTP download by about 4%. Adding gaming and voice cut it by about 60%, mostly due to gaming.
- L4S: while a UDP flow loaded the link, a classic TCP download averaged about 25% under its configured rate with round-trip spikes and retransmissions. The L4S download beside it stayed near its rate with almost no retransmissions.

The overall verdict is FAIL if any single flow misses its loss, latency or MOS threshold.

## What you walk away with

A run produces a verdict, summary numbers and over-time graphs, in forms for people and for machines.

| Output             | Contains                                                                                   | Good for                            |
|--------------------|--------------------------------------------------------------------------------------------|-------------------------------------|
| HTML or PDF report | Overall PASS or FAIL, port setup, per-flow results, graphs over time, latency CDF and CCDF | Reading and sharing a result        |
| JSON report        | The same numbers, structured                                                               | Scripts, dashboards, trend tracking |

| Output                | Contains                                                                  | Good for                                               |
|-----------------------|---------------------------------------------------------------------------|--------------------------------------------------------|
| JUnit XML             | One test result per flow with failure reasons                             | CI systems such as Jenkins or GitLab                   |
| CSV, Excel            | Tabular results from the GUI or command-line runner                       | Spreadsheets                                           |
| Packet capture (pcap) | Raw received frames, filtered                                             | Debugging in Wireshark                                 |
| Live metrics          | Bytes, latency and reordering per flow each second, scraped by Prometheus | Real-time Grafana graphs, very long or very large runs |

What the numbers let you conclude:

- **A capacity figure:** this device forwards X Mbit/s at this frame size with under Y loss.
- **A latency profile:** typical delay, jitter and the worst 1% or 0.1% at a given load.
- **A user-experience estimate:** download speed, call quality score, whether video stalls.
- **A fairness or isolation finding:** what one flow costs another when they share the link.
- **A diagnosis:** TCP graphs separate packet loss from buffer bloat from a badly sized window.

Three ways to drive it: the GUI by hand, a JSON file with the Test Framework for repeatable automated runs, or the Python and Tcl APIs for full control and raw counters. The Reference tab covers each.

## What makes a result untrustworthy

A number is only as good as the mechanism behind it; these are the documented ways each one goes wrong.

| If you see       | Suspect | Because                                                                                                                     |
|------------------|---------|-----------------------------------------------------------------------------------------------------------------------------|
| Negative latency | Clocks  | Latency compares the sender's and receiver's clocks. On two machines they must be continuously synchronised with PTP or NTP |

| If you see                                    | Suspect                | Because                                                                                                                                                             |
|-----------------------------------------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Latency spikes of several seconds             | Corrupted packets      | The timestamp travels in the payload; a damaged one gives a wild value                                                                                              |
| More frames received than sent                | Colliding flows        | Two flows with near-identical frames get counted by each other's filter                                                                                             |
| 100% loss toward a device behind a gateway    | NAT                    | Downstream traffic needs a NAT mapping, which must be opened from inside first                                                                                      |
| Low but stable TCP speed, no retransmissions  | Window scale too small | The test setup, not the device, is the limit                                                                                                                        |
| Good TCP speed with huge round-trip time      | Window scale too large | The sender is queueing in its own buffers                                                                                                                           |
| Different bitrate than expected               | Counting convention    | Rates can count the frame only, frame plus checksum, or everything on the wire (24 more bytes per frame). A 1 Gbit/s link carries only 714 Mbit/s of 60-byte frames |
| A missing latency tail on a 100 Gbit/s system | Sampling               | The 5100 may sample received packets for latency                                                                                                                    |

Limits when the receiving or sending side is an Endpoint on a real device:

- Latency is best-effort. The device clock is set once at registration and can drift, especially on long tests.
- No reordering detection.
- As a receiver it gives average latency only, with no distribution.
- iPhones and iPads never report signal strength.

Every system also has a latency noise floor below which it cannot measure. Excentis publishes it per model, but only as a chart.

## Check yourself

Ten questions covering the notes above; answers follow.

1. Why can frame blasting overload a device while a TCP flow cannot?
2. How does ByteBlower know which received frames belong to which flow?
3. Where does the latency timestamp live, and what does that imply for two machines?
4. Goodput is well below throughput. What is happening?
5. Why does RFC 2544 repeat the search for several frame sizes?
6. In TR-398, what are the two UDP rates and what is the pass line?
7. Which flows never fail a Test Framework run?
8. What MOS does a voice flow need, and what latency rule applies to a UDP flow by default?
9. You measure more frames received than sent. What went wrong?
10. Name three things an Endpoint cannot measure that a server port can.

### Answers

1. Frame blasting sends at a fixed rate regardless of loss. TCP reduces its rate when it sees loss.
2. A receive-side filter (a trigger) unique to each flow counts matching frames.
3. In the last 8 bytes of the frame payload. Sender and receiver clocks must be synchronised, or latency is wrong and can go negative.
4. Many retransmissions, so the path is dropping packets.
5. Small frames stress per-packet processing and large frames stress link capacity, so the limit differs by size.
6. Station 1 at 75% and its partner at 50% of their own TCP maximum; each must keep more than 45% of that maximum, and the sum must exceed the expected total.
7. HTTP (TCP) flows. They are measured and reported only.
8. MOS of at least 4. Loss at most 1%, and when latency analysis is on, P99.9 latency under 5 ms.
9. Two flows have frames too similar to tell apart, so their filters count each other's frames.
10. Reordering, a latency distribution when receiving, and precise latency in general; on iOS also signal strength.